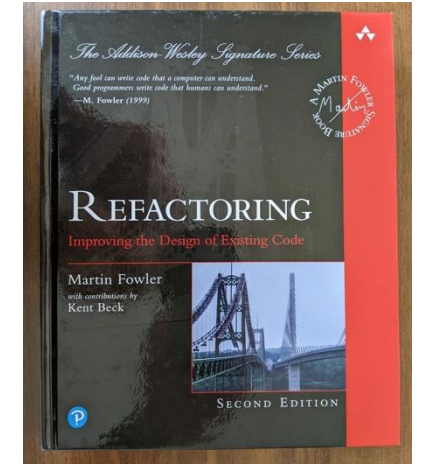


Refactoring

Definition



"Refactoring: to restructure software by applying a series of refactorings without changing its observable behavior."

Martin Fowler (Refactoring, 2nd ed., S. 45)

Voraussetzung für Refactoring

Refactoring setzt voraus, dass **Tests** existieren.

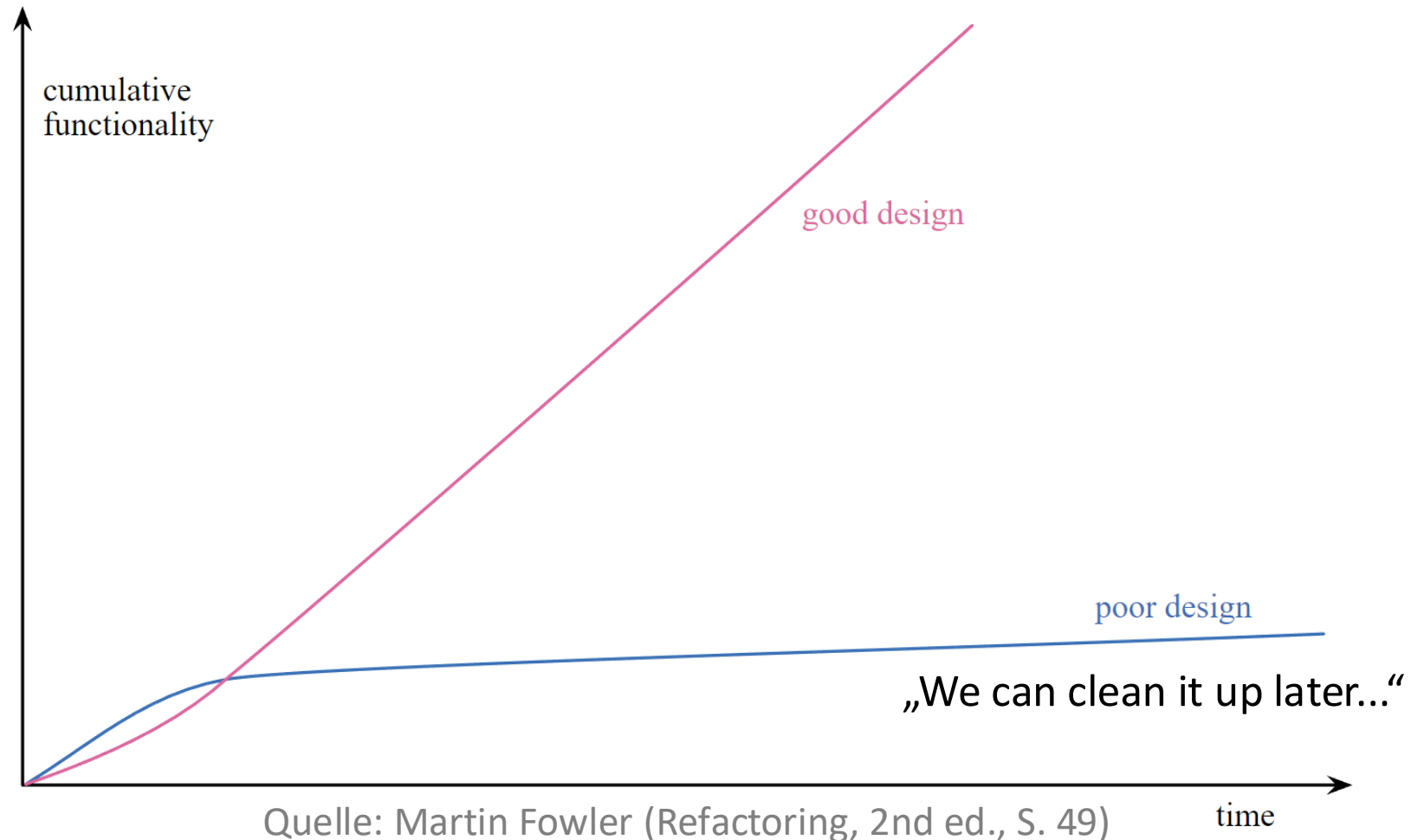
Refactoring soll den Code verbessern, **ohne** das **Verhalten** zu **verändern**.

Das Ziel guten Software-Designs

„We structure our software to make change easier;
after all, software is meant to be soft.“

Martin Fowler (Refactoring, 2nd ed., S. 76)

Das Ziel guten Software-Designs



Siehe auch: „An Empirical Study of Design Degradation: How Software Projects Get Worse over Time”

<https://ieeexplore.ieee.org/document/7321186>

Good Code



Fulfills it's purpose.
Easy to change.

Bad Smells



Ein "Bad Smell" (Codegeruch 🤔) ist ein Anzeichen dafür, das ein tieferliegendes Problem in der Software vorliegt und führt häufig zu "Bad Code".

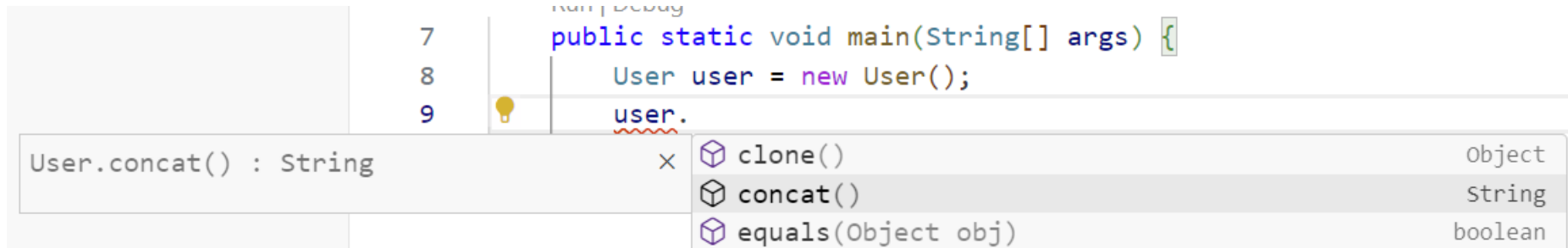
Bad Code:

- Does not do it's job.
- Hard to change.



Mysteriöse Namen: Bad Smell 🍌

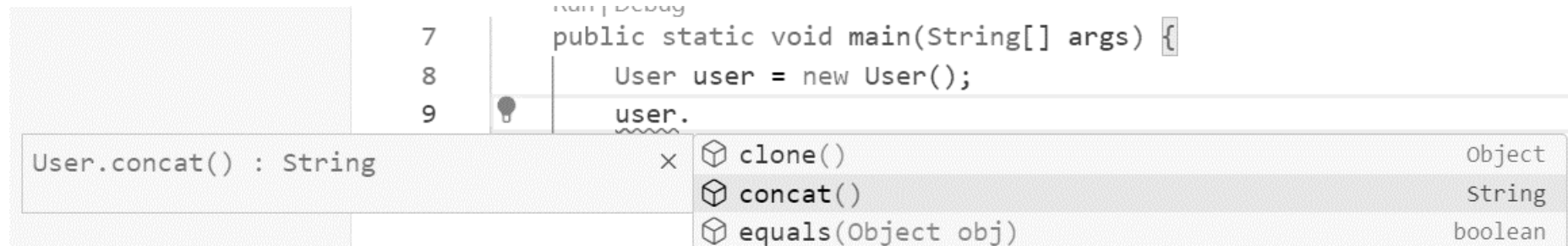
```
public String concat() {  
    return fn + " " + ln;  
}
```



Benenne eine Methode nach dem Zweck und nicht danach, wie sie funktioniert.

Mysteriöse Namen: Refactored

```
public String concat() {  
    return fn + " " + ln;  
}
```



```
public String fullname() {  
    return firstname + " " + lastname;  
}
```

Magic Numbers: Bad Smell

Problem: Was bedeuten die Zahlen?

```
public double calculateTotalPrice(double[] prices) {  
    double totalPrice = 0;  
    for (double price : prices) {  
        if (price > 100) {  
            totalPrice += price * 0.1;  
        } else {  
            totalPrice += price * 0.05;  
        }  
    }  
    return totalPrice;  
}
```

Magic Numbers: Konstanten

Problem: Magic Numbers

Maßnahme: Magic Numbers durch Konstanten oder Variablen ersetzen.

```
public double calculateTotalPrice(double[] prices) {  
    double totalPrice = 0;  
    for (double price : prices) {  
        if (price > PRICE_DISCOUNT_THRESHOLD) {  
            totalPrice += price * HIGH_PRICE_DISCOUNT_RATE;  
        } else {  
            totalPrice += price * LOW_PRICE_DISCOUNT_RATE;  
        }  
    }  
    return totalPrice;  
}
```

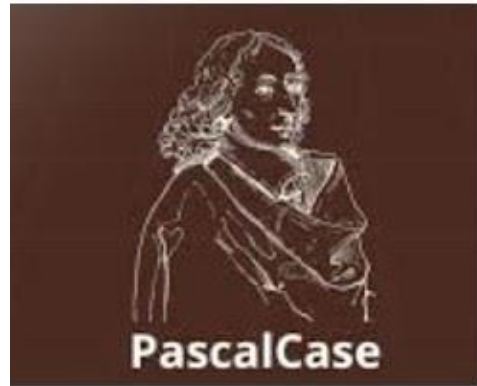


class ClassName

public void methodName

SCREAMING_SNAKE_CASE

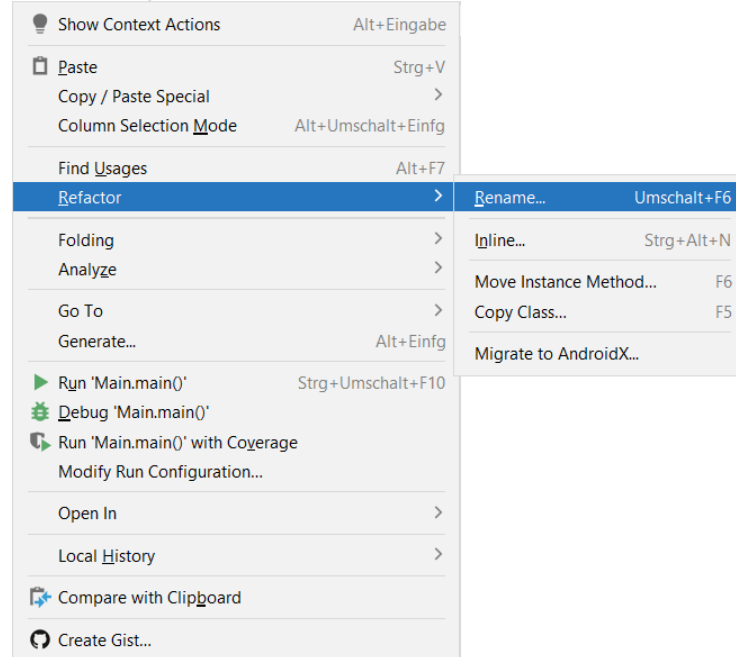
static int *STATIC_VALUE*;



Pro-Tip: Refactor -> Rename

Das Benennen von Klassen, Funktionen und Variablen ist eines der schwierigsten Probleme in der Informatik (durch LLMs einfacher).

Nutze die Rename-Funktion deiner IDE!



Duplizierter Code: Bad Smell 🍌

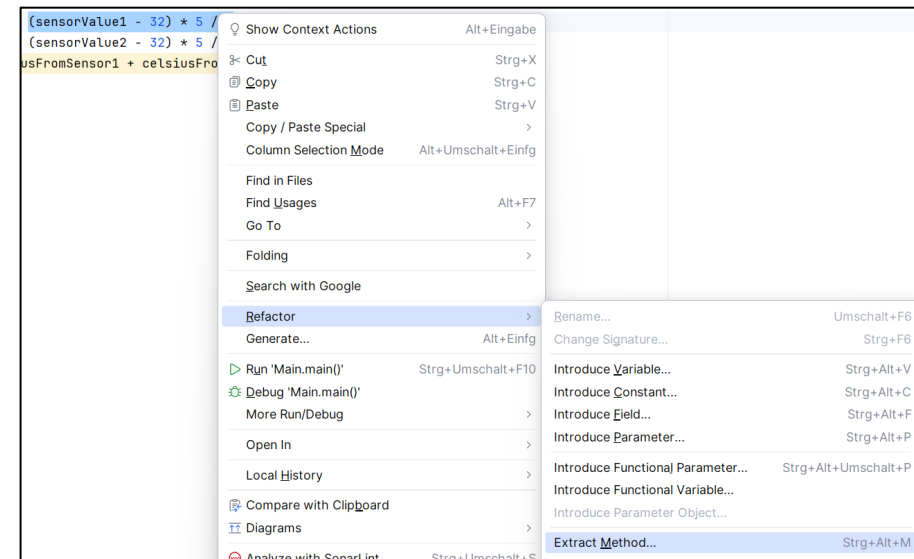
aka *Don't Repeat Yourself* (DRY)

```
double celsiusFromSensor1 = (sensorValue1 - 32) * 5 / 9;  
double celsiusFromSensor2 = (sensorValue2 - 32) * 5 / 9;  
double meanCelsius = (celsiusFromSensor1 + celsiusFromSensor2) / 2;
```

Duplizierter Code: Beispiel

aka *Don't Repeat Yourself* (DRY)

```
double celsiusFromSensor1 = (sensorValue1 - 32) * 5 / 9;  
double celsiusFromSensor2 = (sensorValue2 - 32) * 5 / 9;  
double meanCelsius = (celsiusFromSensor1 + celsiusFromSensor2) / 2;
```



IDE: Extract Method

```
double sumCelsius = fahrenheitToCelsius(sensorValue1) + fahrenheitToCelsius(sensorValue2)  
double meanCelsius = sumCelsius / 2;
```

```
public double fahrenheitToCelsius(double fahrenheit) {  
    return (fahrenheit - 32) * 5 / 9;  
}
```

Kein Separation of Concerns: Bad Smell 🍌

Separation of Concerns hat zum Ziel, verschiedene Aspekte klar voneinander zu trennen, um die **Wartbarkeit**, **Erweiterbarkeit** und **Wiederverwendbarkeit** des Codes zu verbessern.

Das Prinzip besagt, dass Klassen und Methoden so **aufgeteilt** werden sollten, dass sie nur für **eine bestimmte Aufgabe** verantwortlich sind.

Namen mit "And", "Or", "If", "But", "Manager", "Processor", "Handler" deuten oft auf mehrere (zu viele) Aufgaben hin.

Beispiele: ConvertAndFilter, AccountManager, selectAndUpdate(persons);

Maßnahmen:

Aufspalten der Klasse oder Methode (Divide & Impera).



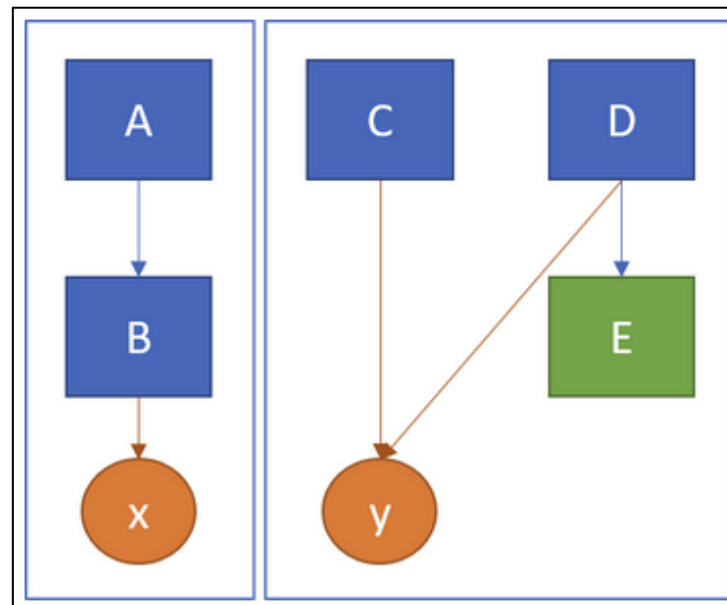
Quelle

Separation of Concerns: Beispiel 1/2

Folgende Klasse hat die **Methoden A bis E** und **Variablen x und y**.

Diese Klasse ließe sich in zwei Klassen aufteilen: **{A, B, x}** und **{C, D, E, y}**.

Klasse(n)

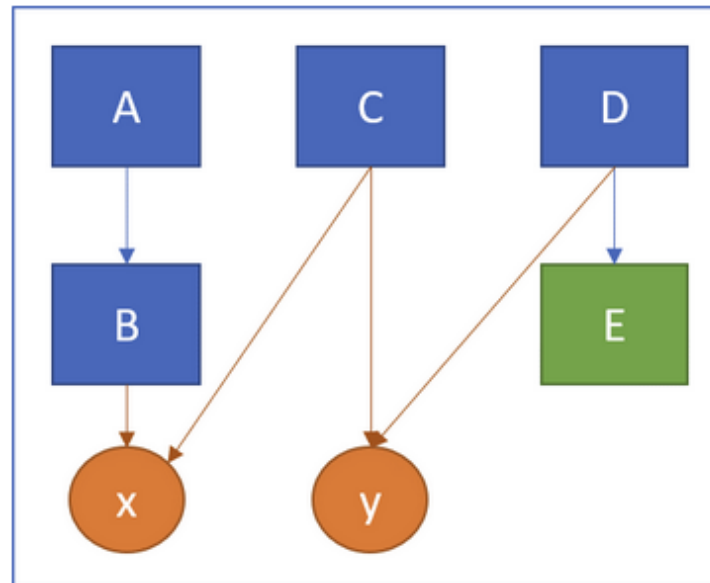


Quelle

Separation of Concerns: Beispiel 2/2

In diesem Beispiel **greift C zusätzlich auf x** zu. Die Klasse lässt sich nicht mehr in zwei Klassen aufteilen.

Klasse



Quelle

Große Klassen: Bad Smell 🍌



Quelle

Zu Große Klassen offenbaren sich häufig durch die Anzahl der Methoden und Attribute.

Ein weiteres Anzeichen: Wenn bestimmte Methoden in einer Klasse immer nur eine Untermenge der Attribute verwenden (siehe Separation of Concerns).

Große Klassen: Daumenregeln

Java-Klassen sollten bis zu 300 *Lines of Code* (LOC) haben.

- Viele Klassen haben oft weniger LOC.
- Ab 1000 LOC sollte man prüfen, ob das seine Richtigkeit hat (das sind häufig automatisch generierte Klassen).

Java-Klassen sollten weniger als 20 öffentliche Methoden haben.

- Jeweils mit bis zu 40 LOC.

Die Anzahl an Attributen einer Klasse sollte minimal sein (das Nötigste).

Feature Neid: Bad Smell 🦌

Feature Neid liegt vor, wenn eine Methode mehr Zeit mit Methoden oder Attributen aus einer anderen Klasse verbringt, als mit den aus der eigenen Klasse.



Maßnahme:

Verschiebe die Methode in die andere Klasse.

Daumenregel:

Lege Dinge zusammen (z. B. in eine Klasse), die regelmäßig gemeinsam verwendet und/oder verändert werden.

Lange Methoden: Bad Smell 🤮

Divide et Impera („Teile und herrsche“)

Grundprinzip: Große oder schwer zu lösende Probleme in kleinere und einfachere aufspalten.

```
120  
121 public Employee GetEmployee(int id, Filter[] filters)  
122 {  
123     Employee e = null;  
124  
125     find employee with filters  
75728  
75729     return e;  
75730 }
```



Quelle

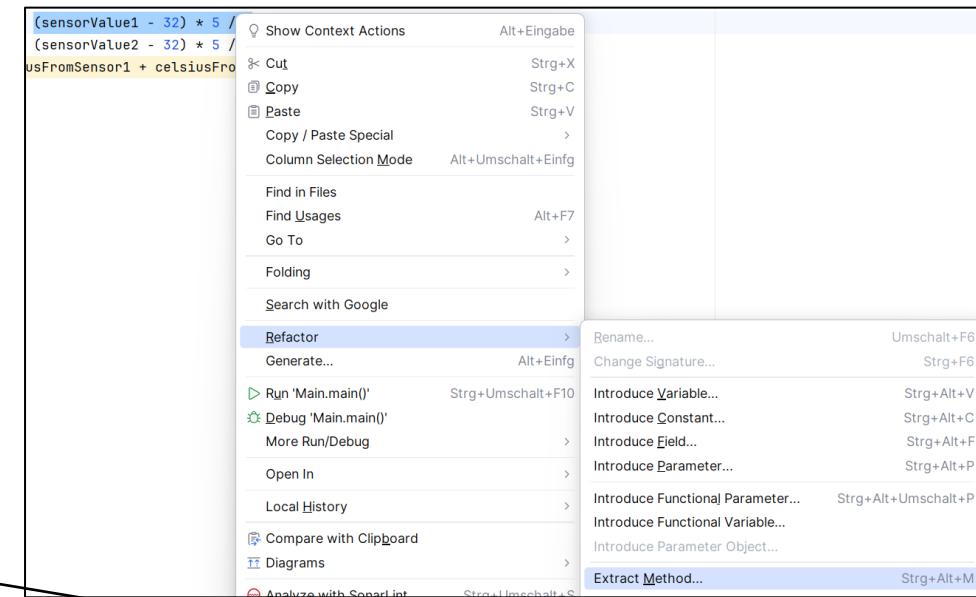
Lange Methoden: Wie lang?



Wie lang sollen Methoden nun sein?

Daumenregel: Möglichst kurz.

nutze



IDE: Extract Method

Maßnahme:

Spalte große Methoden in mehrere kleineren Methoden auf.

Vorteile:

- Kurze Methoden sind oftmals **wiederverwendbarer** und **verständlicher**.
- Kurze Methoden können zu **weiteren Refactoring-Möglichkeiten** führen.
- Kurze Methoden (die keine anderen Methoden aufrufen) lassen sich **leichter testen**.

Refresher: Seiteneffekte

```
int add(int a, int b){  
    return a+ b;  
} // -> Pure Function = Keine Seiteneffekte!
```

```
boolean checkLogin(User user) {  
    if (user.getPassword().equals("1234")) {  
        // Seiteneffekt 1: Wir ändern eine globale Variable  
        globalLoginCounter++;  
  
        // Seiteneffekt 2: Wir schreiben in die Datenbank  
        database.updateLastLogin(user, LocalDateTime.now());  
  
        return true;  
    }  
    return false;  
}  
// -> Hat Seiteneffekte!  
// Die Methode heißt "check...", aber sie tut "write..." und  
"update...".
```

Refresher: Atomare Bedingung

// Ein IF-Statement ... aber wie viele Bedingungen?

```
if (age > 18 && hasLicense) {  
    rentCar();  
}
```

```
if (age > 18) {           // 1. Atomare Bedingung  
    if (hasLicense) {    // 2. Atomare Bedingung  
        rentCar();  
    }  
}
```

Lange Methoden: Zyklomatische Komplexität

Maßnahme: Zyklomatische Komplexität minimieren

Die zyklomatische Komplexität (auch McCabe-Metrik) lässt sich wie folgt ermitteln:

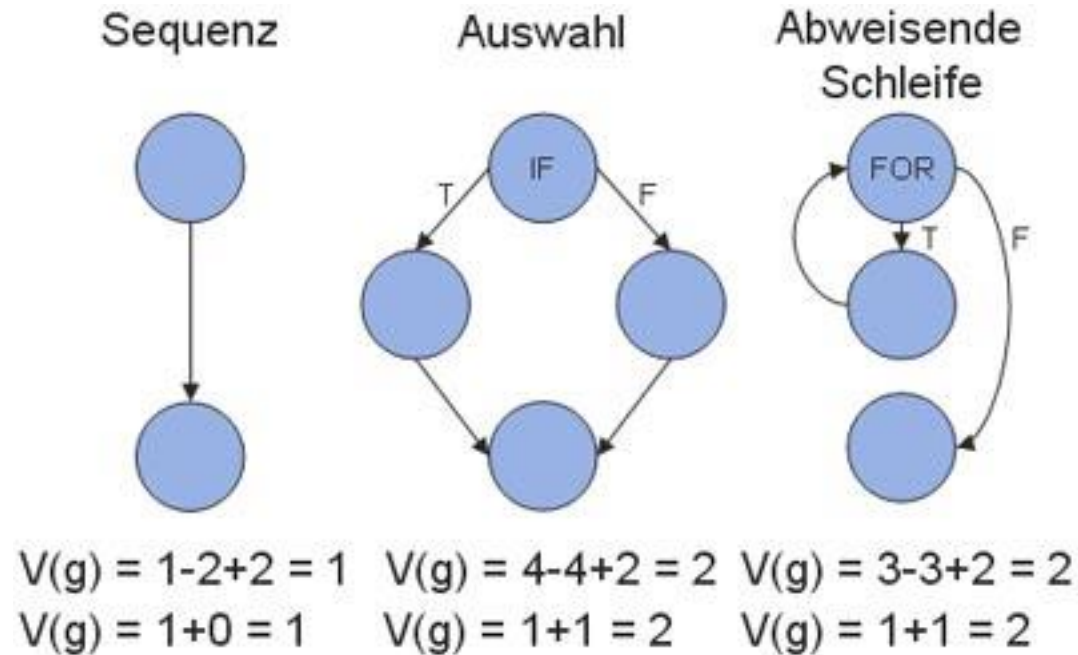
$$Z = b + 1$$

b ist die Anzahl der **atomaren Bedingungen** im Code.

D. h. alle zusammengesetzten Bedingungen (**&&** oder **||** innerhalb von **if**, **while**, **?:**) werden einzeln gezählt. Bei **switch**-Anweisungen wird **jeder case ohne default** gezählt.

FYI: Plugins wie SonarLint berechnen die zyklomatische Komplexität.

Zyklomatische Komplexität: Graphen



Quelle

Zyklomatische Komplexität: Beispiele 1/3

// Keine Bedingungen.

// Z = 0 + 1 = 1

```
int calculate(int value) {  
    int result = 0;  
    result = 2 * value;  
    return result;  
}
```

// Eine atomare Bedingung.

// Z = 1 + 1 = 2

```
int calculate(int value) {  
    int result = 0;  
    if (value % 2 == 0) {  
        result = 2 * value;  
    }  
    return result;  
}
```

// Eine atomare Bedingung.

// Z = 1 + 1 = 2

```
int calculate(int value) {  
    int result = 0;  
    if (value % 2 == 0) {  
        result = 2 * value;  
    } else {  
        result = 3 * value;  
    }  
    return result;  
}
```

// Zwei atomare Bedingungen.

// Z = 2 + 1 = 3

```
int calculate(int value) {  
    int result = 0;  
    if ((value % 2 == 0) && (value > 0)) {  
        result = 2 * value;  
    } else {  
        result = 3 * value;  
    }  
    return result;  
}
```

Zyklomatische Komplexität: Beispiele 2/3

```
// Z = ?  
int calculate(int value) {  
    if (value < 35) {  
        return 1 * value;  
    } else if (value < 50) {  
        return 2 * value;  
    } else if (value < 100) {  
        return 3 * value;  
    } else {  
        return 4 * value;  
    }  
}
```

```
// Z = ?  
int calculate(int value) {  
    int result = 0;  
    for (int i = 0; i < value; i++) {  
        result = 1 * value;  
    }  
    int i = 0;  
    while (i < value) {  
        result = 2 * value;  
    }  
    do {  
        result = 3 * value;  
    } while (i < value);  
    return result;  
}
```

```
// Z = ?  
int greatestCommonDivisor(int x, int y) {  
    while (x != y) {  
        if (x > y) {  
            x -= y;  
        } else {  
            y -= x;  
        }  
    }  
    return x;  
}
```

Zyklomatische Komplexität: Beispiele 3/3

// Drei atomare Bedingungen.

// $Z = 3 + 1 = 4$

```
int calculate(int value) {  
    if (value < 35) {  
        return 1 * value;  
    } else if (value < 50) {  
        return 2 * value;  
    } else if (value < 100) {  
        return 3 * value;  
    } else {  
        return 4 * value;  
    }  
}
```

// Drei atomare Bedingungen.

// $Z = 3 + 1 = 4$

```
int calculate(int value) {  
    int result = 0;  
    for (int i = 0; i < value; i++) {  
        result = 1 * value;  
    }  
    int i = 0;  
    while (i < value) {  
        result = 2 * value;  
    }  
    do {  
        result = 3 * value;  
    } while (i < value);  
    return result;  
}
```

// Zwei atomare Bedingungen.

// $Z = 2 + 1 = 3$

```
int greatestCommonDivisor(int x, int y) {  
    while (x != y) {  
        if (x > y) {  
            x -= y;  
        } else {  
            y -= x;  
        }  
    }  
    return x;  
}
```

Zyklomatische Komplexität: Semantik

Laut [McCabe](#) sollte die **zyklomatische Komplexität** eines in sich abgeschlossenen Teilprogramms (Komponente) **nicht höher als 10 sein**, da sonst das Programm **zu komplex und zu schwer zu testen** ist.

1-10: Simple procedure, little risk

11-20: More complex, moderate risk

21-50: Complex, high risk

>50: Untestable code, very high risk



Zyklomatische Komplexität: Aussagekraft?

Die zyklomatische Komplexität ist **umstritten**, da sie sich nur erhöht, wenn verzweigende Anweisungen wie if eingefügt werden, aber nicht beim Hinzufügen sonstiger Anweisungen.

Die zyklomatische Komplexität kann eine **Aussage über den Testaufwand** treffen, d. h. die Anzahl der zu testenden unabhängigen Programmpfade bzw. der **benötigten Testfälle**.

// Z = 7 + 1 = 8

```
String weekdayName(int number) {  
    return switch (number) {  
        case 1 -> "Monday";  
        case 2 -> "Tuesday";  
        case 3 -> "Wednesday";  
        case 4 -> "Thursday";  
        case 5 -> "Friday";  
        case 6 -> "Saturday";  
        case 7 -> "Sunday";  
        default -> "Unknown";  
    };  
}
```

// Z = 2 + 1 = 3

```
String weekdayName(int number) {  
    String[] days = new String[]{"Monday", "Tuesday", "Wednesday",  
        "Thursday", "Friday", "Saturday", "Sunday"};  
  
    String result = "Unknown";  
    if ((number >= 1) && (number <= days.length)) {  
        result = days[number - 1];  
    }  
    return result;  
}
```

FYI: Mit der Halstaed-Metrik lässt sich die Komplexität berechnen.

Zyklomatische Komplexität: Reduktion

Problem: Hohe Zyklomatische Komplexität

Maßnahmen:

Die zyklomatische Komplexität lässt sich durch **Hilfsmethoden, Schleifen, HashMaps, weniger Verschachtelung, und Polymorphismus** reduzieren.



Jedes switch oder if/else if(x.equals(y))/else lässt sich durch eine Map oder durch Polymorphismus ersetzen (und umgekehrt)!

Zyklomatische Komplexität: if/else?

```
public void executeCommand(String command) {  
    if ("START".equals(command)) {  
        System.out.println("Starting server...");  
        bootSystem();  
    } else if ("STOP".equals(command)) {  
        System.out.println("Stopping server...");  
        shutdownSystem();  
    } else if ("RESTART".equals(command)) {  
        System.out.println("Rebooting...");  
        shutdownSystem();  
        bootSystem();  
    } else {  
        throw new IllegalArgumentException("Unknown  
command: " + command);  
    }  
}
```

Zyklomatische Komplexität: map!

```
public class CommandHandler {  
    // Die Map hält die Zuordnung von "Name" -> "Logik"  
    private static final Map<String, Runnable> COMMAND_MAP  
= new HashMap<>();
```

```
    // Initialisierung (könnte auch im Konstruktor sein)
```

```
    static {  
        COMMAND_MAP.put("START", () -> {  
            System.out.println("Starting server...");  
            bootSystem();  
        });  
        COMMAND_MAP.put("STOP", () -> {  
            System.out.println("Stopping server...");  
            shutdownSystem();  
        });  
        COMMAND_MAP.put("RESTART", () -> {  
            System.out.println("Rebooting...");  
            shutdownSystem();  
            bootSystem();  
        });  
    }  
}
```

```
    // Die eigentliche Methode
```

```
    public void executeCommand(String command) {  
        // Kein if, kein switch. Nur ein Lookup.  
        Runnable action = COMMAND_MAP.get(command);  
  
        if (action == null) {  
            throw new IllegalArgumentException("Unknown  
command: " + command);  
        }  
        action.run();  
    }  
}
```

Lange Parameterlisten: Bad Smell 🦌

Je **weniger Parameter**, desto einfacher ist es, eine Methode zu **verstehen, aufzurufen** und zu **testen**.

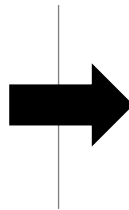
Spätestens **ab drei Parameter** fängt es an zu riechen.

Lange Parameterlisten: Methoden aufteilen

Wenn eine Methode zu viele Parameter hat, kann dies ein Hinweis darauf sein, dass sie zu viele Aufgaben erledigt. In diesem Fall sollte die Methode in kleinere, spezifischere Methoden aufgeteilt werden, die jeweils weniger Parameter benötigen.

Maßnahme: Aufteilen von Methoden

```
trainClassifiers(true);  
trainClassifiers(false);
```



```
trainClassifiersMultiThreaded();  
trainClassifiersSingleThreaded();
```

Methoden mit Flag-Parametern (Boolean-Werte) verstoßen häufig gegen das Prinzip: „Separation of Concerns“.

Lange Parameterlisten: Parameter-Objekt

Beispiel: `showGraph(customer, unit, null, true, false);`



Maßnahme: Parameter-Objekt

```
var settings = new Settings();  
settings.setUnit("cm");  
settings.setComment(null);  
settings.setCompactMode(true);  
settings.setVerbose(false);  
showGraph(customer, settings);
```

// Alternative mit Method Chaining:

```
var settings = new Settings()  
    .setUnit("cm")  
    .setComment(null)  
    .setCompactMode(true)  
    .setVerbose(false);  
showGraph(customer, settings);
```

ABER: Die Maßnahme reduziert nicht den Testaufwand.



Method Chaining

```
class Settings {  
  
    // Einzelne Einstellungen (Attribute) werden mit Default-Werten initialisiert,  
    // wodurch nicht jeder einzelne Wert gesetzt werden muss (optionale Settings).  
    private String unit = "cm";  
    private String comment = "";  
    private boolean compactMode = false;  
    private boolean verbose = false;  
  
    // Jeder Setter gibt statt klassisch void das Setting-Objekt selbst zurück.  
    Settings setUnit(String unit) {  
        this.unit = unit;  
        return this;  
    }  
  
    Settings setComment(String comment) {  
        this.comment = comment;  
        return this;  
    }  
  
    Settings setCompactMode(boolean compactMode) {  
        this.compactMode = compactMode;  
        return this;  
    }  
  
    Settings setVerbose(boolean verbose) {  
        this.verbose = verbose;  
        return this;  
    }  
}
```

Method Chaining, auch als Fluid API bezeichnet, ist eine Technik, bei der Methodenaufrufe hintereinander verkettet werden, sodass der Code flüssig und lesbar wirkt. Bekannte Beispiele sind Streams `filter(...).map(...).toList();` Mockito `when(...).thenReturn(...);` und `Optional<T> map(...).orElse(...);`

```
record Customer(String name) {}  
  
class Demo {  
  
    public static void main(String[] args) {  
        Customer customer = new Customer("Elon");  
  
        // Old-school Variante ohne Method Chaining:  
        Settings settings = new Settings();  
        settings.setUnit("cm");  
        settings.setComment("elon!");  
        settings.setCompactMode(true);  
        settings.setVerbose(false);  
  
        // Method chaining:  
        // Each setter method returns the current Setting-object.  
        // Advantage: less parameters for showGraph and setter calls can be optional.  
        settings = new Settings()  
            .setUnit("cm")  
            .setComment("elon!")  
            .setCompactMode(true)  
            .setVerbose(false);  
  
        showGraph(customer, settings);  
    }  
  
    static void showGraph(Customer customer, Settings settings) {  
        // use customer and settings...  
    }  
}
```

Lange Parameterlisten: Parameter-Attribut

(Variation zu Parameter-Objekt)

Beispiel: `showGraph(customer, unit, null, true, false);`



Maßnahme: Parameter-Objekt als Attribut

```
public class Statistics {  
    private final Settings settings;  
    public Statistics(Settings settings) { // Dependency Injection  
        this.settings = settings;  
    }  
    // Parameter lassen sich (teilweise oder komplett) durch Attribute ersetzen.  
    public void showGraph(Customer customer) {  
        // ... use settings attribute ...  
    }  
}
```

ABER: Die Maßnahme reduziert nicht den Testaufwand.

Diese Variante bietet sich an, wenn `showGraph` immer wieder mit demselben `Settings`-Objekt aufgerufen wird.

Hier wird das `Settings`-Objekt nur einmalig im Konstruktor übergeben und dann in `showGraph` wiederverwendet.

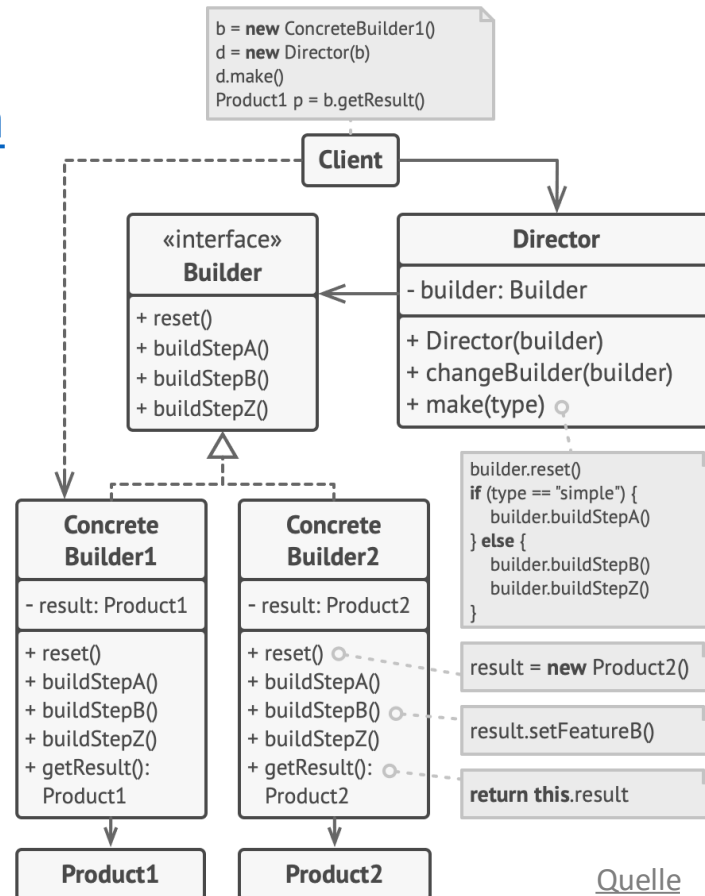
Lange Parameterlisten: Builder-Pattern

Beispiel: `showGraph(customer, unit, null, true, false);`

Maßnahme: Builder-Pattern



Sneak Preview:
Software Engineering
nächstes Semester!



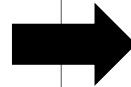
Quelle

Tiefe Verschachtelung: Bad Smell 🍌

Halte die Verschachtelungstiefe auf ein Minimum. Häufig lässt sich durch geschicktes Umdrehen der Logik die Verschachtelung reduzieren und auch das else vermeiden.

```
Customer customer = database.findCustomer("Elon");
```

```
if (customer != null) {  
    Address address = customer.address();  
    if (address != null) {  
        System.out.println(address);  
    } else {  
        System.err.println("No address found!");  
    }  
} else {  
    System.err.println("No address found!");  
}
```



```
Customer customer = database.findCustomer("Elon");
```

```
// Siehe auch: Fail Fast Principle
```

```
if (customer == null || customer.address() == null) {  
    System.err.println("No address found!");  
}  
System.out.println(customer.address());
```

Meidung vorhandener Interfaces: Bad Smell

Verwende möglichst **Interfaces für Parameter (und Rückgabewerte)**:

`double calcAverage( LinkedList<Student> students);` // Konkrete Klasse statt Interface.

`double calcAverage(Collection<Student> students);` // calcAverage ist flexibler aufrufbar.

Die Methode mit Collection als Parameter lässt sich viel flexibler aufrufen als die Methode mit LinkedList.

Die Methode: Input und Output

Output (Rückgabewert)



Input (Parameterliste)



```
public Document appendFooter(Document document) throws Exception  
{ ... }
```

Methodensignatur = Methodenname + Parameterliste + Exceptions

Parameter als Return-Wert: Bad Smell

In- und Output?

```
appendFooter(document);
```



Output

```
var newDocument = appendFooter(document);
```



Input

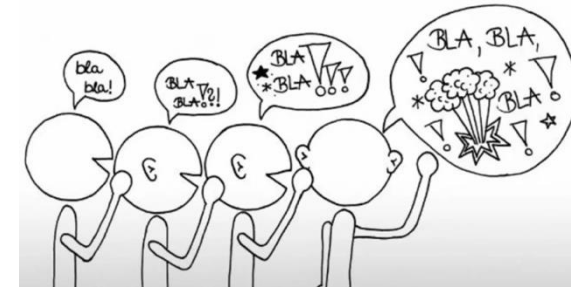
Verwende **Parameter** möglichst **nicht als Output**: d. h. modifiziere Parameter nicht innerhalb einer Methode, sondern gebe stattdessen einen neuen Wert bzw. neues Objekt als Rückgabewert zurück.

Minimiere dadurch **Seiteneffekte**! Je weniger Seiteneffekte, desto weniger Bugs!



Nachrichtenketten: Bad Smell 🍌

aka Law of Demeter



Quelle

Wenn Methodenaufrufe miteinander verkettet werden, um an bestimmte Objekte zu gelangen, ist dies häufig ein Bad Smell.

Beispiel: **user.getInfo().getContactInfo().getPhoneNumber();**

Dieses "Durchhangeln" verstößt gegen die Kapselung und gibt unnötig die interne Datenstruktur der Klasse preis, die außerhalb der Klasse nicht bekannt sein sollte.

Maßnahme: **user.getPhoneNumber();**

Globale Daten: Bad Smell

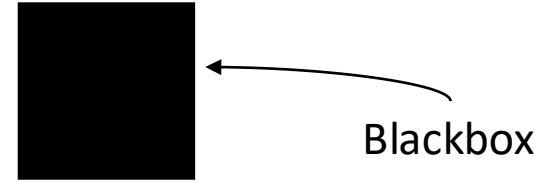
Globale Daten sind Variablen, die von (fast) **überall** im Code **gelesen** und/oder **geändert** werden können (auch Singletons).

Globale Daten sind eine Quelle für **Seiteneffekte** und **Bugs**. 

Sie machen Code unwartbar, schwer nachvollziehbar, kaum testbar.



Globale Daten: Kapselung



Globale Daten sind eine Quelle für Seiteneffekte und Bugs.

Sie machen Code unwartbar, schwer nachvollziehbar, kaum testbar.

Maßnahme: Nutze Kapselung

Gebe jedem Sprachelement die möglichst niedrigste Sichtbarkeit.

`private` -> `protected` -> `package` -> `public`

Veränderbare (mutable) Daten: Bad Smell 🤮

Veränderbare Daten sind eine Quelle für **Seiteneffekte** und **Bugs**,  insbesondere wenn sie **von mehreren Objekten verwendet** werden.

Beispiele: mutable Listen, Setter-Methoden, public Attribute

Veränderbare Daten können Code unwartbar, schwer nachvollziehbar, kaum testbar machen. Außerdem sind sie **schlechter parallelisierbar**.

In der objektorientierten Welt lassen sich veränderbare Daten nicht komplett vermeiden, aber man sollte sie auf ein **Minimum reduzieren**.

Veränderbare (mutable) Daten: Immutable

Maßnahme #1: Verwende Immutable Data

```
return Collections.unmodifiableCollection(result); // siehe auch List.of oder List.copyOf
```

```
return result.stream().filter(...).map(...).toList(); // erzeugt eine unmodifiableCollection
```

```
record Customer(String firstName, String lastName) {} // Attribute sind nicht änderbar
```

```
// Erstelle neue Objekte, statt sie zu modifizieren (vermeide Setter-Methoden)
```

```
var updatedCustomer = new Customer(customer.firstName(), newLastName);
```

Veränderbare (mutable) Daten: Scope 1/2

Maßnahme #2: Minimiere Scope

Halte die Reichweite (Scope) von Daten so gering wie möglich.
Ziehe z. B. lokale Variablen gegenüber Attributen vor (sofern möglich).

Bevorzugte Reihenfolge:

Code-Block {...} -> Methode -> Klasse -> Package -> Global

```
var product = products.find(id);  
if(updateProduct) {  
    update(product);  
} else {  
    add(new Product(name, price);  
}
```



```
if(updateProduct) {  
    var product = products.find(id); // Scope für product so gering wie möglich (hier: Code-Block {})  
    update(product);  
} else {  
    add(new Product(name, price)); // Die Variable product wird im else-Fall nicht benötigt.  
}
```

Veränderbare (mutable) Daten: Scope 2/2

Maßnahme #2: Minimiere Scope

Halte die Reichweite (Scope) von Daten so gering wie möglich.
Ziehe z. B. lokale Variablen gegenüber Attributen vor (sofern möglich).

Bevorzugte Reihenfolge:

Code-Block {...} -> Methode -> Klasse -> Package -> Global

```
public static void main(String[] args) {  
    String firstName, lastName, age;  
    boolean isSpecial;  
    Scanner scanner = new Scanner(System.in);  
    System.out.println("Enter your name: ");  
    firstName = scanner.next();  
    lastName = scanner.next();  
    // ...  
}
```

Old-School C-Style



```
public static void main(String[] args) {  
    System.out.println("Enter your name: ");  
    Scanner scanner = new Scanner(System.in);  
    String firstName = scanner.next();  
    String lastName = scanner.next();  
    // ...  
}
```

Primitive Besessenheit: Bad Smell 🍌



Quelle

Programmiersprachen bestehen oft aus primitiven Datentypen wie `int`, `float`, `double` und `char`.

Es ist erlaubt, eigene Datentypen zu definieren (aka **strongly typed identifier**).

aka Wrapper-Klasse

Beispiele:

```
record Money(double value, Currency currency) {...}
```

```
record PhoneNumber(String number) {...}
```

```
record MustacheFile(String filename) { ... }
```

Vorteile: Mehr Semantik, Validierung und Datenzusammenhalt möglich.

Nachteile: Mehr Abstraktion und mögliche null-Objekte.

Spekulative Abstraktion: Bad Smell

"Oh, wir werden eines Tages bestimmt diesen Spezialfall haben, den implementiere ich schnell. Besser auf alle Eventualitäten vorbereitet sein..."

Programmiere im Hier und Jetzt, nicht in der Zukunft! 

Keep it simple stupid (KISS)

Maßnahmen:

Klassen und Methoden, die nichts zu tun haben, wegwerfen.

Unnötige Indirektion entfernen: $A \rightarrow B \rightarrow C$ wird zu $A \rightarrow C$.

Ungenutzten Code / Attribute / Parameter entfernen.



Bla

Kommentare: Bad Smell 🍌

Kommentare sind fast immer ungepflegt, veraltet, redundant und falsch.

Kommentiere nur das, was sich in der Programmiersprache nicht direkt ausdrücken lässt (z. B. Meta-Informationen, Geschäftsregeln, externe Limitierungen).

Die besten "Kommentar"-Funktionen sind:

Extract Method und **Introduce Variable**



Kommentare: Beispiel

Mythos:

"Kommentiere so viel wie möglich."



Problem:

Die Kommentare sind sinnlos und teilweise falsch.

```
/**
 * Manages boardgames.
 */
class BoardGameManager {

    /**
     * Storage for the managed board games.
     */
    private List<BoardGame> boardgames = new ArrayList<>();

    /**
     * The logger implementation used by this manager.
     */
    private Log logger = null; // init with default value.

    /**
     * Adds a boardgame to this manager.
     *
     * @param name the name of the added boardgame.
     */
    public boolean add(Boardgame boardgame) {
        ...
        return true; // returns true
    }
}
```



Kommentare: Refactored

Mythos:
Kommentiere so
viel wie möglich.



```
class BoardGameManager {  
  
    private List<BoardGame> boardgames = new ArrayList<>();  
  
    private Log logger = null;  
  
    public boolean add(Boardgame boardgame) {  
        ...  
        return true;  
    }  
}
```



Kommentare: Extract Method 1/3

2 usages

```
private Employee employee;
```

1 usage

```
private String notice;
```

no usages

```
public String generatePensionNotice(String[] args) {  
    // Check if the employee is eligible for full benefits  
    if ((employee.getFlag() == HOURLY_FLAG) && (employee.getAge() > 65)) {  
        // create notice ...  
    }  
    return notice;  
}
```



Kommentare: Extract Method 2/3

2 usages

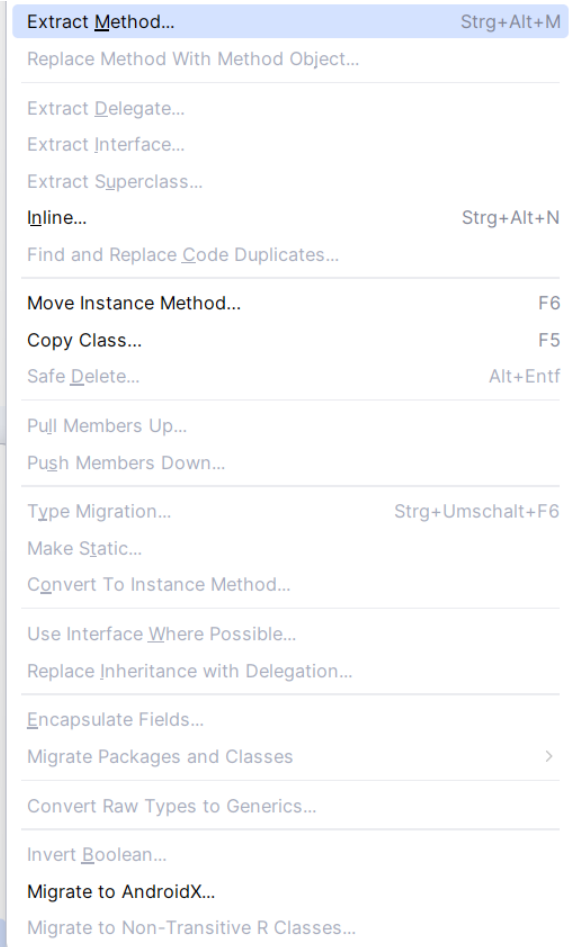
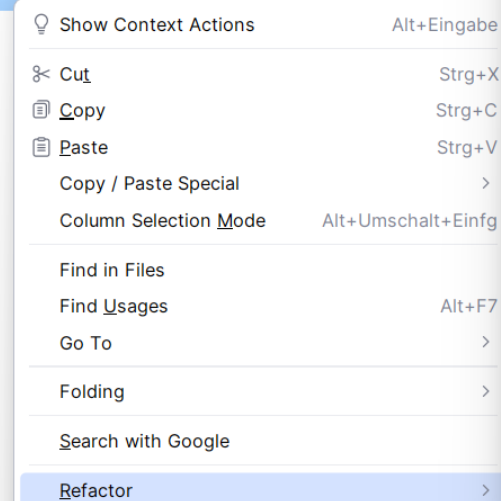
```
private Employee employee;
```

1 usage

```
private String notice;
```

no usages

```
public String generatePensionNotice(String[] args) {  
    // Check if the employee is eligible for full benefits  
    if ((employee.getFlag() == HOURLY_FLAG) && (employee.getAge() > 65)) {  
        // create notice ...  
    }  
    return notice;  
}
```



Kommentare: Extract Method 3/3

Vorher

```
public String generatePensionNotice(String[] args) {  
    // Check if the employee is eligible for full benefits  
    if ((employee.getFlag() == HOURLY_FLAG) && (employee.getAge() > 65)) {  
        // create notice ...  
    }  
}
```



Nachher

no usages

```
public String generatePensionNotice(String[] args) {  
    if (checkIfEmployeeIsEligibleForFullBenefits()) {  
        // create notice ...  
    }  
    return notice;  
}
```

1 usage

```
private boolean checkIfEmployeeIsEligibleForFullBenefits() {  
    return ((employee.getFlag() == HOURLY_FLAG) && (employee.getAge() > 65));  
}
```

Das Ergebnis ohne Kommentar nach Ausführung von Extract Method.
Der Kommentar wurde buchstäblich durch die Methode ersetzt.

Kommentare: Code-Leichen



// Fixes uneven steps of timestamps

// result = createFixedStepTimestamps(result, sampleRateInMs);

result = updateBoschImuSensitivity(result);



Auskommentiert. Aber ist der Code noch wichtig?!



Merke: Lösche auskommentierten Code und nutze Git, um alten Code zu speichern und anzuschauen.

Checked Exceptions: Bad Smell 🍌

Verwende unchecked
[RuntimeException](#) statt
checked Exceptions
wie z. B. IOException.

(außer es gibt einen Grund,
für checked Exceptions, z. B.
besondere
Sicherheitsanforderungen)

```
class UserRepo {  
    1 usage  
    public void save(User user) throws IOException {  
        Files.write(Path.of(user.name()), user.toString().getBytes());  
    }  
}  
  
no usages  
@Controller  
public class UserController {  
    no usages  
    @GetMapping("/users")  
    public String listUsers() {  
        UserRepo repo = new UserRepo();  
        repo.save(new User( name: "admin"));  
        return "U";  
    }  
}
```

Unhandled exception: java.io.IOException

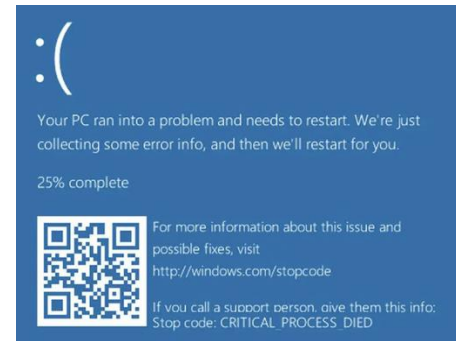
Add exception to method signature Alt+Umschalt+Eingabe More actions... Alt+Eingabe

© UserRepo

```
public void save(  
    @NotNull User user  
)  
throws IOException
```

Throws: IOException

P2-ZP2



Error Codes: Bad Smell 🤮

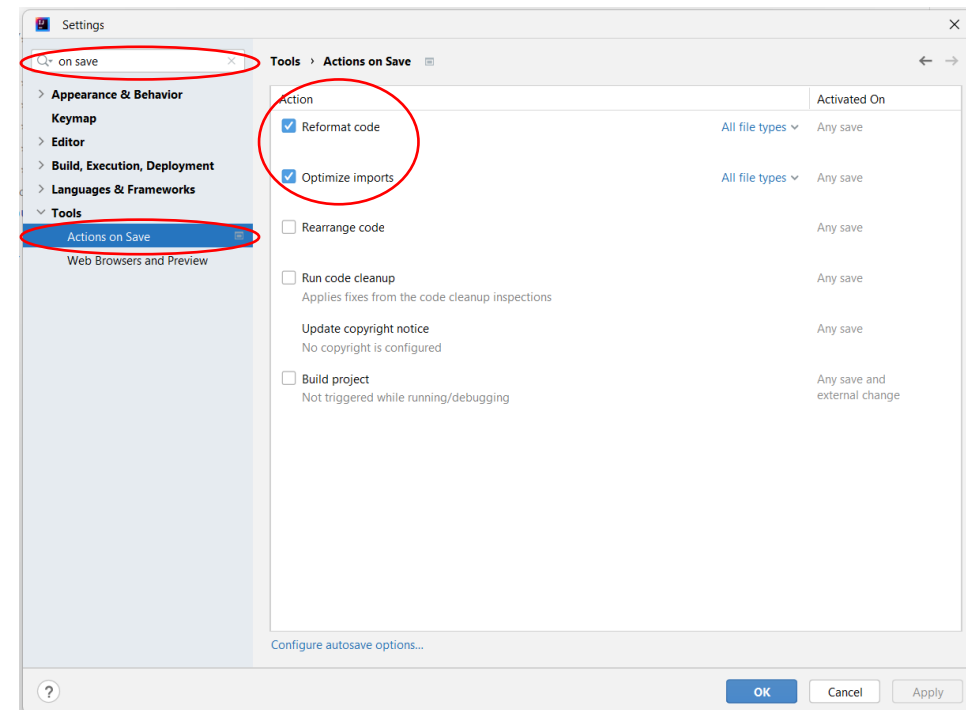
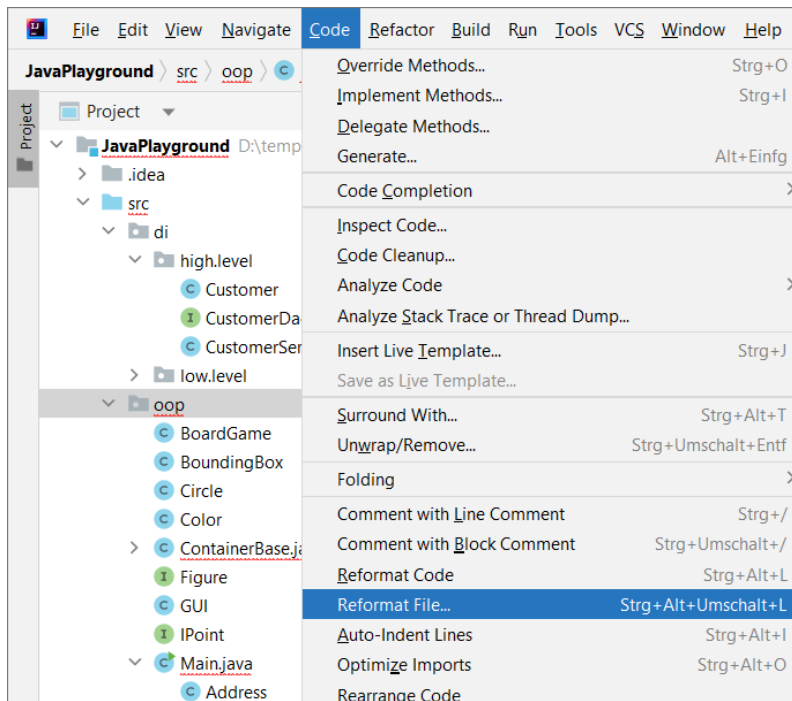
Vermeide Error Codes, verwende stattdessen RuntimeException.

```
if (deletePage(page) == E_OK) {  
    if (registry.deleteReference(page.name) == E_OK) {  
        if (configKey.deleteKey(page.name.makeKey()) == E_OK) {  
            logger.log("page deleted");  
        } else {  
            logger.log("configKey not deleted");  
        }  
    } else {  
        logger.log("deleteReference from registry failed");  
    }  
} else {  
    logger.log("delete failed");  
    return E_ERROR;  
}
```

```
try {  
    deletePage(page);  
    registry.deleteReference(page.name);  
    configKeys.deleteKey(page.name.makeKey());  
} catch (Exception e) {  
    logger.log(e.getMessage());  
    // handle exception ...  
}
```

Keine Code-Formatierung: Bad Smell 🍌

1. Verwende **im Team** einen **einheitlichen** Formatierungsstandard.
2. Nutze die **Auto-Code-Formatierung** der Entwicklungsumgebung.



Code Review

Nutze im Teams Code Reviews, zum Beispiel mittels [GitHub](#) Pull-/Merge-Requests, um Feedback zu erhalten / geben.

```
211 211      - (void)fetchImageForKey(NSString *)key
212 212      {
213 213          [super fetchImageForKey:key];
214 214      -
215 215      +
216 216      +      dispatch_async(dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_HIGH, 0), ^{
217 217      +          Image *image = [_imageManager fetchImageForKey:key];
218 218      +          _imageView.image = image;
```



pullrequest bot 34 minutes ago

You should not access UI elements like this imageView outside of the main thread as it will lead to intermittent crashes in your app.

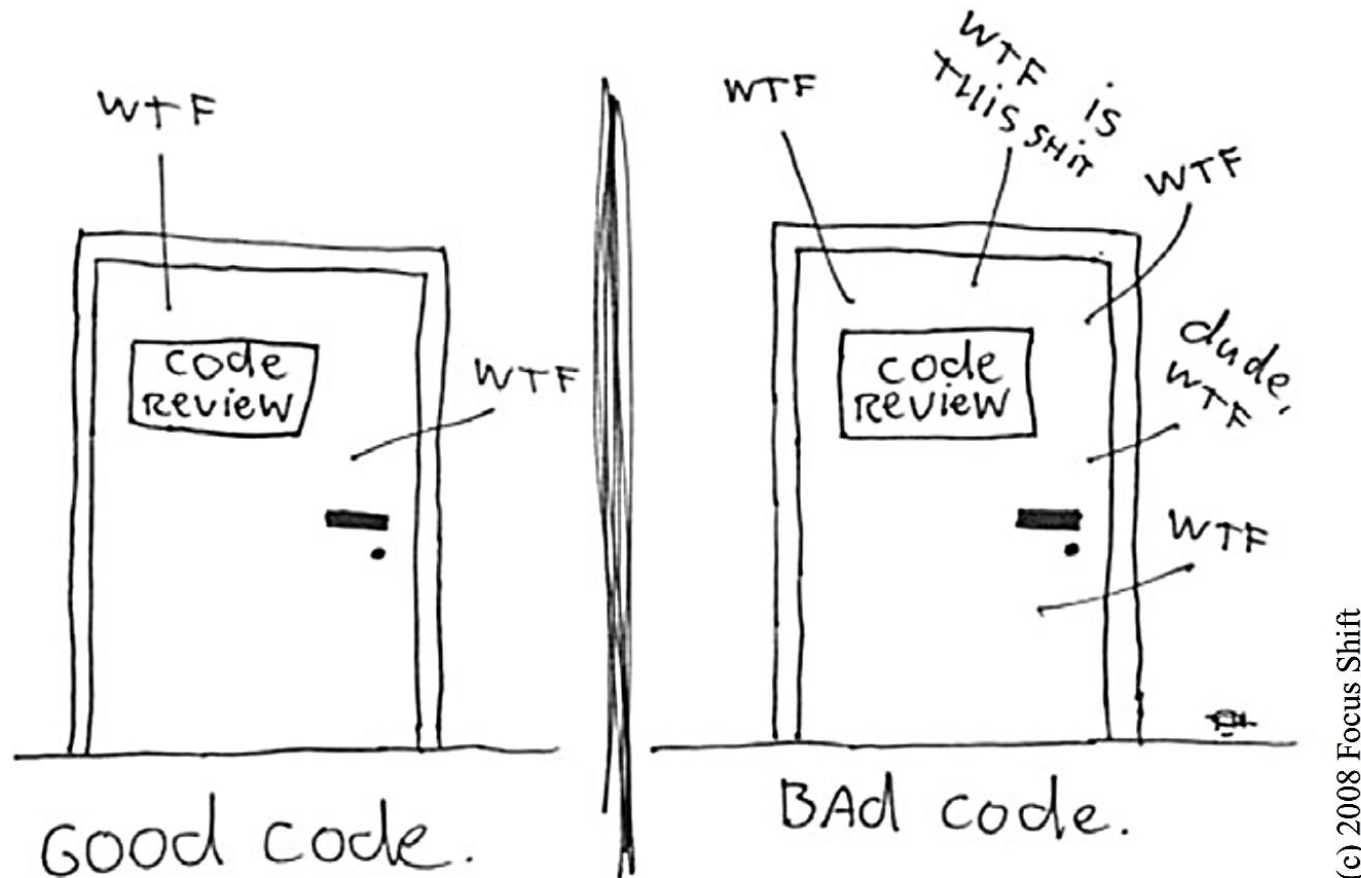


Reply...

Resolve conversation

Code Review: WTFs/Minute

The ONLY VALID MEASUREMENT
OF CODE QUALITY: WTFs/minute



(c) 2008 Focus Shift

Reproduced with the kind permission of Thom Holwerda.
http://www.osnews.com/story/19266/WTFs_m

Tools

[SonarLint](#) ist eine IDE-Erweiterung. Es erkennt und markiert Probleme, die zu Bugs, Schwachstellen und Bad Smells führen können:

- [IntelliJ](#)
- [Visual Studio Code](#)
- [Eclipse](#)

FYI: [Checkstyle](#) ist eine Alternative, die sich mehr auf Formatierung fokussiert.



Refactoring vs Performance

In der Regel sind es fast immer wenige **Flaschenhalse** im Code, die die Performance wesentlich bestimmen.

Oftmals sind nur 10 % des Codes für 90 % der Performance zuständig.
Flaschenhalse zu finden und zu optimieren, bringt daher am meisten Performance.
FYI: Tools zum [Profiling](#) helfen Flaschenhälse aufzuspüren.

Abhängig vom Softwareprojekt kann Code-Optimierung zunächst oft vernachlässigt werden. Typische Ausnahmen: Datenbankzugriffe, Games, Echtzeit-Anwendungen.

Merke: Wenn Performance keine Anforderung ist, sollte nicht-optimierter Code vorgezogen werden, der dafür lesbarer und wartbarer ist.

Refactoring vs Performance: Vergleich

Keine Methoden und Hilfsvariablen

```
public class RunWithoutCleanCode {
    private static final int MANY_TIMES = 100000000;
    private static final StringBuilder outputBuffer = new StringBuilder();
    private final Random random = new Random();

    public static void main(String[] args) {
        new RunWithoutNestedMethods().calculatePrice();
    }

    private void calculatePrice() {
        Instant start = Instant.now();
        for (int i = 0; i < MANY_TIMES; i++) {
            var orderQuantity = random.nextInt(0, 1000);
            var orderPrice = random.nextDouble(0, 1000);
            var price = orderQuantity * orderPrice - Math.max(0, orderQuantity - 500) *
                orderPrice * 0.05 + Math.min(orderQuantity * orderPrice * 0.1, 100);
            String formattedPrice = String.format("%.2f", price);
            String priceWithCurrency = formattedPrice + " Euro";
            outputBuffer.append(priceWithCurrency + " Euro");
        }
        Instant finish = Instant.now();

        long timeElapsed = Duration.between(start, finish).toMillis();
        System.out.println();
        System.out.println(timeElapsed + " ms"); // 42883 ms
    }
}
```

42883 ms

VS

Viele Methoden und Hilfsvariablen

```
public class RunWithCleanCode {
    private static final int MANY_TIMES = 100000000;
    private static final StringBuilder outputBuffer = new StringBuilder();
    private final Random random = new Random();

    public static void main(String[] args) {
        new RunWithNestedMethods().calculatePrice();
    }

    private void calculatePrice() {
        Instant start = Instant.now();
        for (int i = 0; i < MANY_TIMES; i++) {
            Order order = createRandomOrder();
            double price = getPrice(order);
            displayFormattedPrice(price);
        }
        Instant finish = Instant.now();
        printElapsedTime(start, finish); // 41180 ms
    }

    private static void printElapsedTime(Instant start, Instant finish) {
        long timeElapsed = Duration.between(start, finish).toMillis();
        System.out.println();
        System.out.println(timeElapsed + " ms");
    }

    private Order createRandomOrder() {
        var order = new Order(random.nextInt(0, 1000), random.nextDouble(0, 1000));
        return order;
    }
}
```

41180 ms

Der Java-Compiler optimiert viele kleine Methoden und Objekte, sodass diese keine negativen Auswirkungen auf die Performance haben.

Performance



Grundregel:
Measure, don't guess!

Selbststudium

- Clean Code (Robert C. Martin, ISBN: 978-0-13-235088-4)
- Effective Java 3rd ed. (Joshua Bloch, ISBN: 978-0-13-468599-1)
- Refactoring 2nd ed. (Martin Fowler, ISBN: 978-0-13-475759-9)
- Langlebige Software-Architekturen 3rd. ed. (Carola Lilienthal, ISBN: 978-3-86490-729-6)

